

8.1 Basic Data Structures

A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently.

The study of data structures leads from traditional structures to the **object-oriented paradigm**. 面向对象范式 (OOP)

Arrays and Aggregates

Array

An array is a rectangular block of data whose entries are all of the same type.

One-dimensional Array $A[0], A[1], A[2], \dots, A[n]$

Two-dimensional Array $A[\text{row}][\text{column}]$

Aggregate type 聚合类型

An aggregate type is a block of data items that may have different types and sizes.

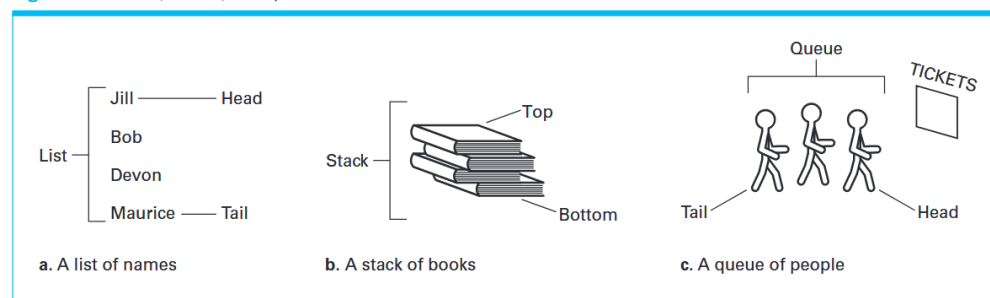
-**Fields** The individual items within an aggregate type are called fields.

aggregate = container

fields = 内部属性

Lists, Stacks, and Queues

Figure 8.1 Lists, stacks, and queues



List: A list is a collection whose entries are arranged sequentially.

-**Head** The beginning of a list is called the head.

-**Tail** The other end of a list is called the tail.

Common operations on lists include inserting, removing, processing, rearranging, and searching entries.

Stack A stack is a list in which entries are inserted and removed only at the head.

-**Top** The head of a stack is called the top.

-**Bottom/Base** The tail of a stack is called the bottom or base.

-**Pushing** Inserting an entry at the top of a stack is called pushing.

-**Popping** Removing an entry from the top of a stack is called popping.

-**last-in (first-out LIFO)** A stack follows the Last-In, First-Out (LIFO) principle.

-**Backtracking** Stacks are often used in backtracking, where actions are reversed in the opposite order from which they were performed.

栈常用于回溯 (backtracking), 即按照与执行顺序相反的顺序撤销操作。

Queue

-**first-in first-out (FIFO)** A queue follows the First-In, First-Out (FIFO) principle.

Queue as Buffer(缓冲区): Queues are used as buffers to store data temporarily during transfer.

Trees

Tree: A tree is a collection whose entries have a hierarchical organization.

Node Each position in a tree is called a node.

-root node The node at the top of the tree is called the root node.

-terminal nodes (leaf nodes) Nodes at the bottom with no children are called terminal nodes or leaf nodes.

Depth The depth of a tree is the number of nodes in the longest path from the root to a leaf.

Children The immediate descendants(子孙,后代) of a node are called its children.

Parent The immediate ancestor of a node is called its parent.

Siblings(兄弟节点) Nodes that share the same parent are called siblings.

binary tree A binary tree is a tree in which each node has at most two children.

-subtrees A subtree is a node together with all the nodes below it.

-branch Every node in a tree can be considered the root of a subtree.

Figure 8.2 An example of an organization chart

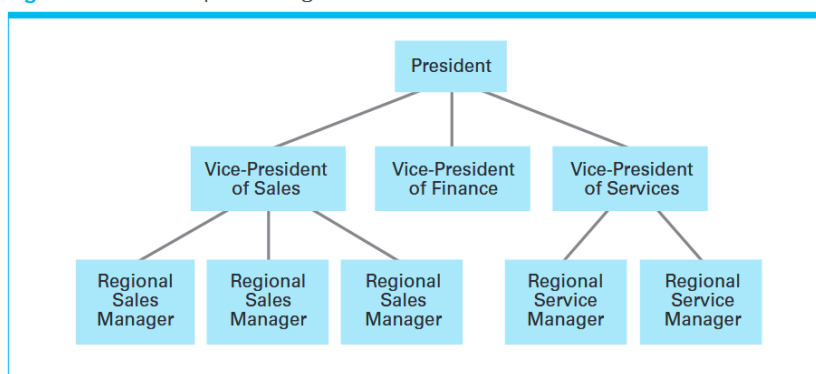
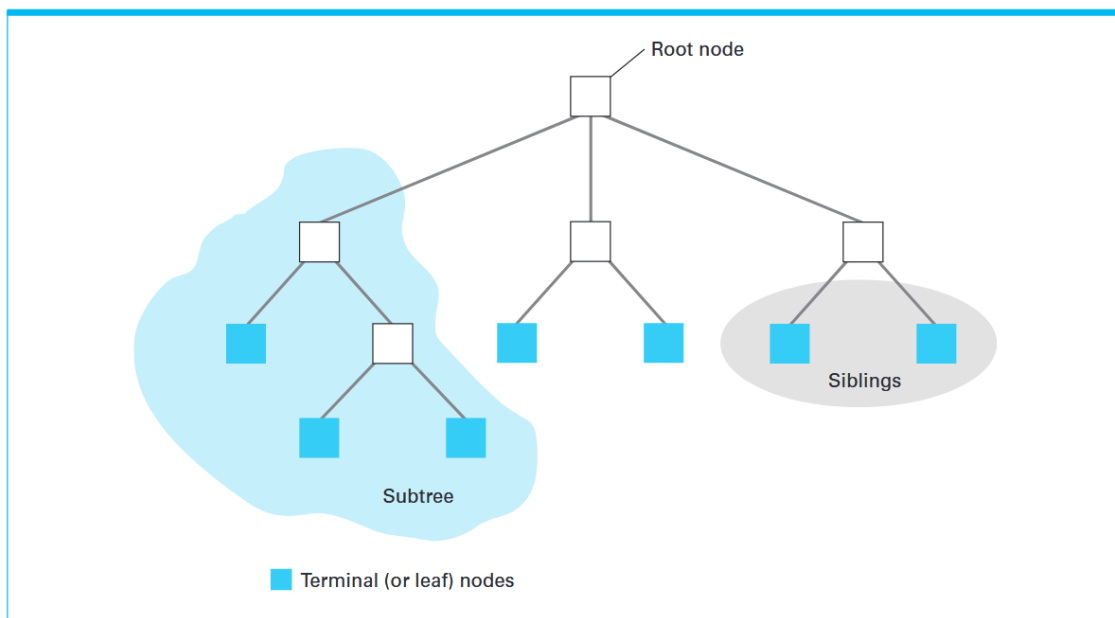


Figure 8.3 Tree terminology



1. Give examples (outside of computer science) of each of the following structures: list, stack, queue, and tree.

List - a shopping list (购物清单)

Stack - a stack of plates (一叠盘子)

Queue - people waiting in line (排队)

Tree - a family tree (族谱)

2. Summarize the distinction between lists, stacks, and queues.

Lists 没什么顺序 elements are in order, but no restriction on access

Stacks 后进先出 last-in, first-out (LIFO)

Queues 先进先出 first-in, first-out (FIFO)

- 3. Suppose the letter A is pushed onto an empty stack, followed by the letters B and C, in that order. Then suppose that a letter is popped off the stack and the letters D and E are pushed on. List the letters that would be on the stack in the order they would appear from top to bottom. If a letter is popped off the stack, which letter will be retrieved?**

E <- top

D

B

A

If a letter is popped off the stack, E letter will be retrieved.

- 4. Suppose the letter A is placed in an empty queue, followed by the letters B and C, in that order. Then suppose that a letter is removed from the queue and the letters D and E are inserted. List the letters that would be in the queue in the order they would appear from head to tail. If a letter is now removed from the queue, which letter will it be?**

B C D E

If a letter is now removed from the queue, B will it be.

- 5. Suppose a tree has four nodes A, B, C, and D. If A and C are siblings and D's parent is A, which nodes are leaf nodes? Which node is the root?**

B

^

A C

|

D

D C nodes are leaf nodes.

B is the root.

8.2 Related Concepts

Abstraction Again

Abstraction is the process of using simplified models to represent complex systems.

Abstraction allows users to interact with data structures without knowing their internal implementation, thereby simplifying system design and improving modularity(模块化程度).

Static Versus Dynamic Structures

Static Structure: A static structure is one whose size or shape does not change over time.

Dynamic Structure: A dynamic structure is one whose size or shape can change over time.

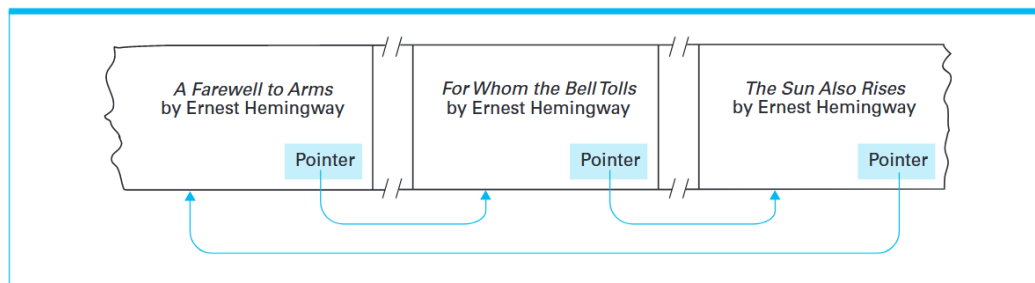
Static structures have fixed size and are easier to manage, while dynamic structures can change size but require more complex memory management.

Pointers

Each cell in main memory is identified by a numeric address.

Memory addresses can be encoded and stored in memory cells because they are numeric values.

Figure 8.4 Novels arranged by title but linked according to authorship



pointer

A pointer is a storage area that contains an encoded memory address.

Pointers are used to record the location where data items are stored.

Dynamic Tracking: A pointer can be updated when data is moved to a new location.

Data can be accessed by following the pointer to its stored address.

instruction pointer

An instruction pointer is a register that holds the address of the next instruction to be executed.

The instruction pointer directs the CPU to the next instruction.

1. In what sense are data structures such as arrays, lists, stacks, queues, and trees abstractions?

Data structures such as arrays, lists, stacks, queues, and trees are abstractions because they allow users to access and manipulate data without knowing how the data is actually stored in memory.

2. Describe an application that you would expect to involve a static data structure. Then describe an application that you would expect to involve a dynamic data structure.

a static data structure: An example is a class roster with a fixed number of students, where the size does not change.

a dynamic data structure: An example is a contact list in a mobile phone, where entries can be added or removed over time.

3. Describe contexts outside of computer science in which the pointer concept occurs.

Outside computer science, the pointer concept appears in situations where one item refers to the location of another. For example, a page number in a book points to where information is located, and a signpost points to a destination.

8.3 Implementing Data Structures

Storing Arrays

Arrays are stored in consecutive memory cells.

The address of the first element is called the **base address**. x

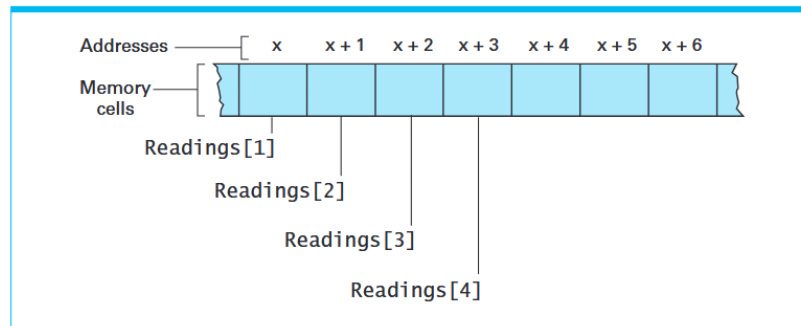
One-dimensional Array Address Calculation

The address of the i th element is computed as:

$$x + (i - 1)$$

Example: The fourth element is stored at address $x + (4 - 1)$.

Figure 8.5 The array of temperature readings stored in memory starting at address x



Two-dimensional Array Storage

A two-dimensional array is stored in a contiguous block of memory.

Arrays are static structures with fixed size.

row major order

Row major order stores the array row by row.

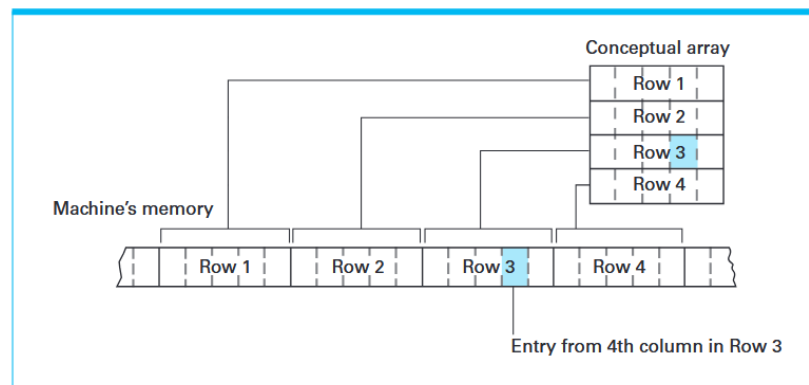
All elements of the first row are stored first, followed by the second row, and so on.

column major order

Column major order stores the array column by column.

Row major order stores rows first, while column major order stores columns first.

Figure 8.6 A two-dimensional array with four rows and five columns stored in row major order



address polynomial

The expression used to compute the memory address is called the address polynomial.

In row major order, the address of the element in row i and column j is:

$$x + (c \times (i - 1)) + (j - 1)$$

Example:

```
int Sales[8, 5]; Sales[3, 4] = 5;
```

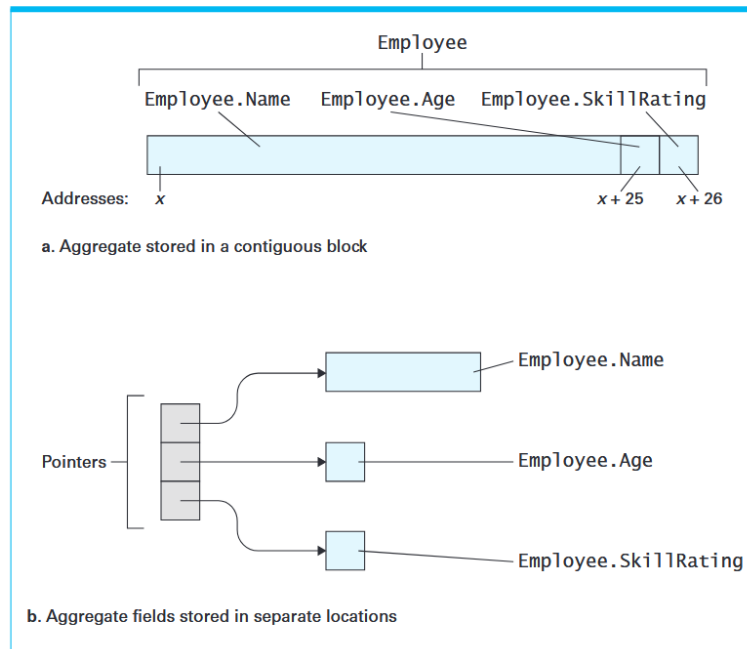
For $c = 5$, $i = 3$, $j = 4$:

$$\text{Address} = x + (5 \times (3 - 1)) + (4 - 1) = x + 13$$

Storing Aggregates

An aggregate(结构体) is a collection of fields that may have different data types.

Figure 8.7 Storing the aggregate type Employee



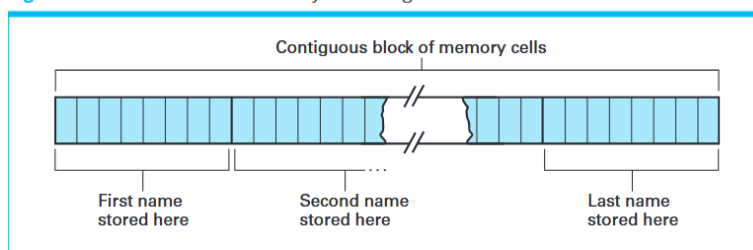
Storing Lists

contiguous list: A contiguous list stores all entries in a single block of consecutive memory cells.

Each entry is stored in a fixed-size subblock(固定大小的小块) within the larger block.

K	i	t	t	y
M	a	r	r	y
J	a	c	k	/space
J	o	e	/space	/space

Figure 8.8 Names stored in memory as a contiguous list



Advantage: Contiguous lists are simple and easy to access.

Disadvantage: Insertion and deletion require shifting elements. / The entire list may need to be moved to a new memory location.

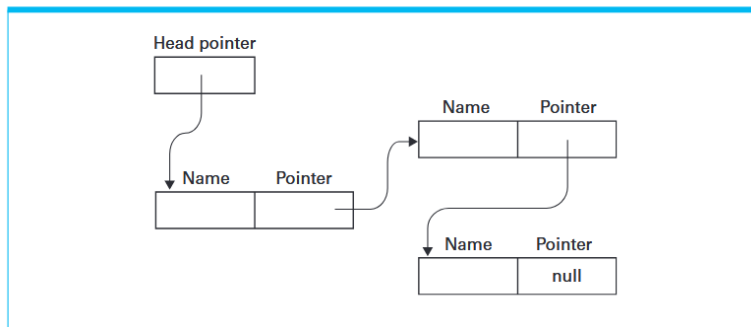
linked list

A **linked list** stores entries in separate memory locations connected by pointers.

Structure: Each entry contains data and a pointer to the next entry.

Entries are stored in separate blocks scattered in memory.

Figure 8.9 The structure of a linked list



head pointer

A head pointer stores the address of the first entry in the list.

null pointer (NIL pointer)

A null pointer indicates the end of the list.

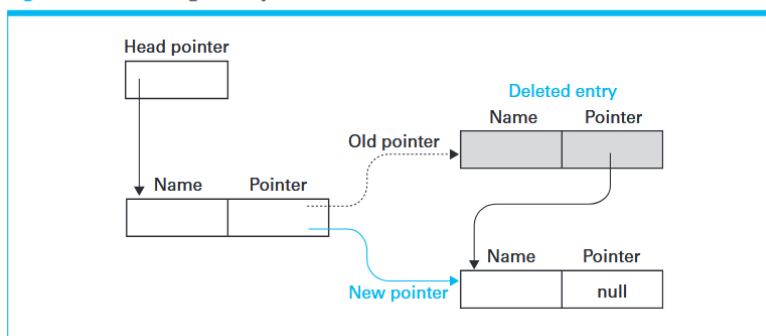
It is a special value that does not point to any valid memory location.

Traversal (遍历)

Traversal starts from the head pointer and follows pointers until the null pointer is reached.

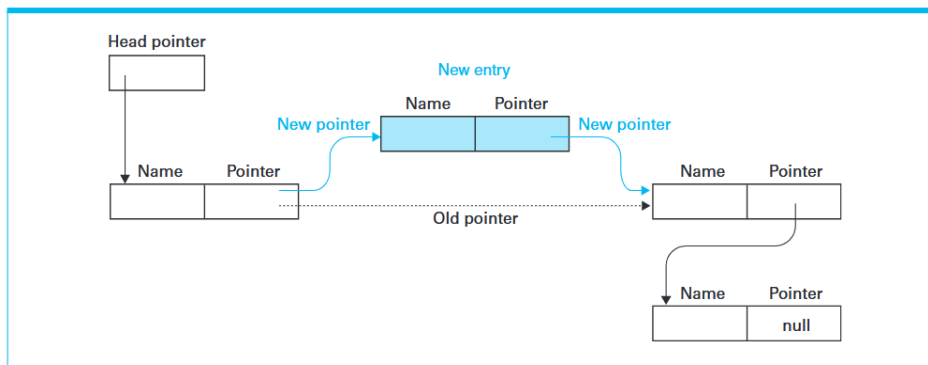
Deletion (删除) In a linked list, deletion is done by changing a pointer.

Figure 8.10 Deleting an entry from a linked list



Insertion (插入) Insertion involves allocating a new block and updating pointers.

Figure 8.11 Inserting an entry into a linked list



Storing Stacks and Queues

Storing Stacks

A stack is stored in a contiguous block of memory.

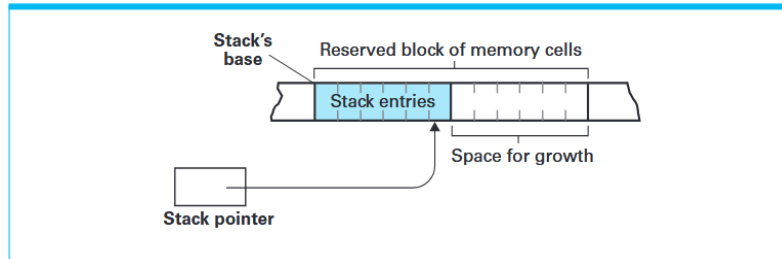
Base of Stack: One end of the block is designated as the base of the stack.

Stack Pointer: The stack pointer stores the address of the top of the stack.

Push Operation (压栈) : To push, first move the stack pointer to the next free location, then store the new entry.

Pop Operation (出栈) : To pop, read the value at the stack pointer, then move the pointer down.

Figure 8.12 A stack in memory



Queues

A queue is also stored in a contiguous block of memory.

head pointer

The head pointer points to the first entry in the queue.

tail pointer

The tail pointer points to the next available position at the end.

Empty Queue (空队列) When the queue is empty, head and tail pointers point to the same location.

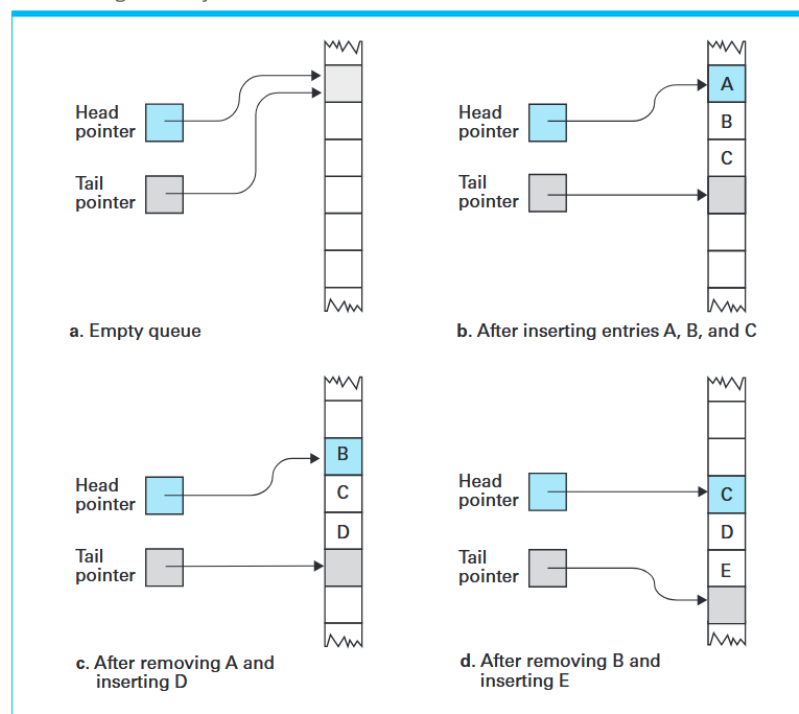
Enqueue (入队) Insert the new entry at the tail pointer, then move the tail pointer forward.

Dequeue (出队) Remove the entry at the head pointer, then move the head pointer forward.

Memory Waste Problem (问题) As elements are inserted and removed, the queue moves through memory.

Consequence (后果) Unused space accumulates at the beginning of the block.

Figure 8.13 A queue implementation with head and tail pointers. Note how the queue crawls through memory as entries are inserted and removed



circular queue

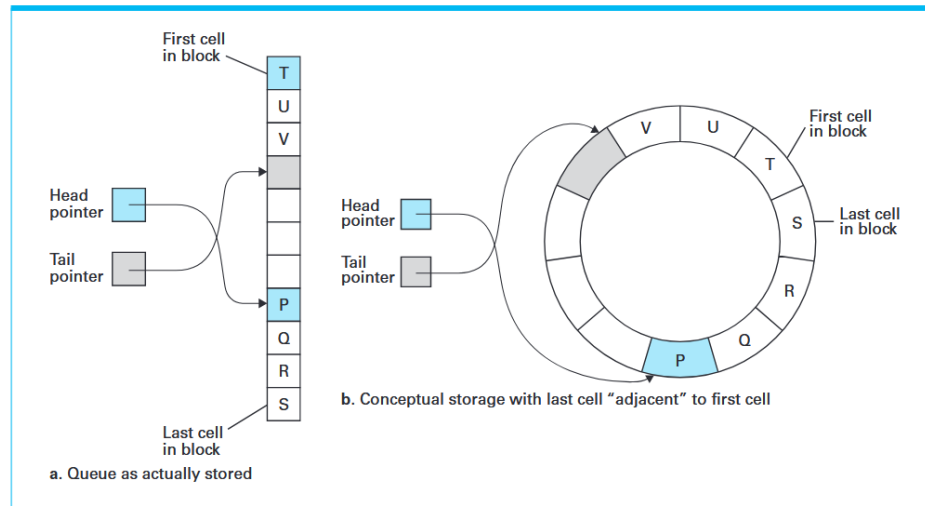
A circular queue treats the memory block as if its ends are connected.

Wrap Around (循环) When the tail reaches the end, it wraps around to the beginning.

Head Movement (头指针) The head pointer also wraps around when needed.

Advantage A circular queue prevents memory waste and uses space efficiently.

Figure 8.14 A circular queue containing the letters P through V



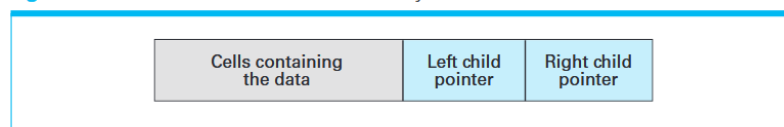
Storing Binary Trees

Binary Tree Node Structure (二叉树节点结构)

Each node in a binary tree has three components:

- Data
- left child pointer
- right child pointer

Figure 8.15 The structure of a node in a binary tree



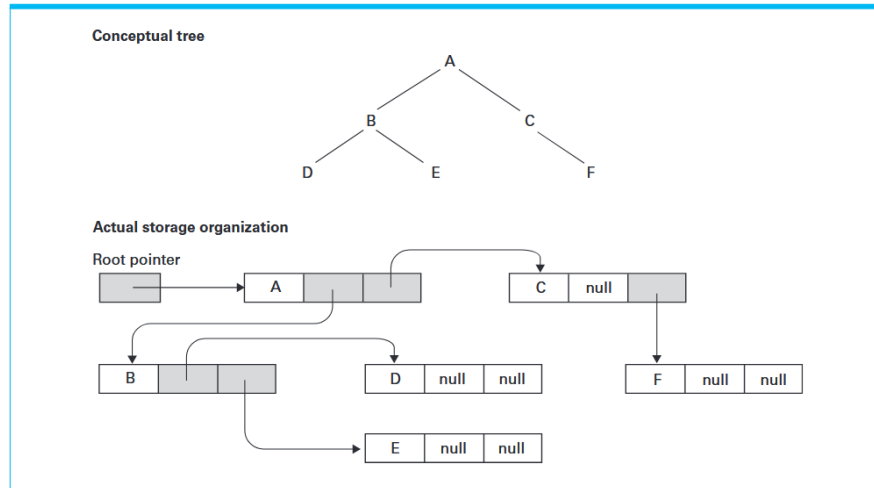
Linked Storage (链式存储) Nodes are stored in memory as separate blocks and linked via their left and right child pointers.

Terminal Node (终端节点) A terminal node has both pointers set to null.

root pointer (根指针) A root pointer is a special memory location that stores the address of the root node.

Traversal in Linked Structure (链式遍历) Start at the root pointer, then follow left or right child pointers to traverse the tree.

Figure 8.16 The conceptual and actual organization of a binary tree using a linked storage system



Contiguous Block Storage (连续块存储)

An alternative is storing the entire tree in a single contiguous block of memory.

Figure 8.17 A tree stored without pointers

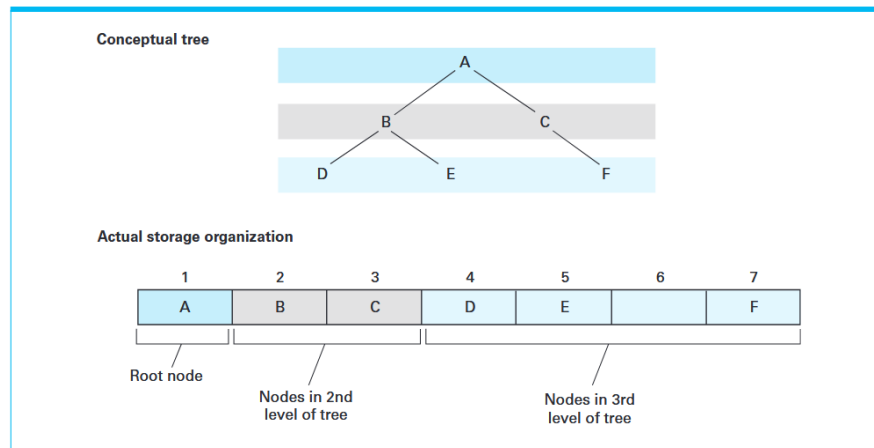
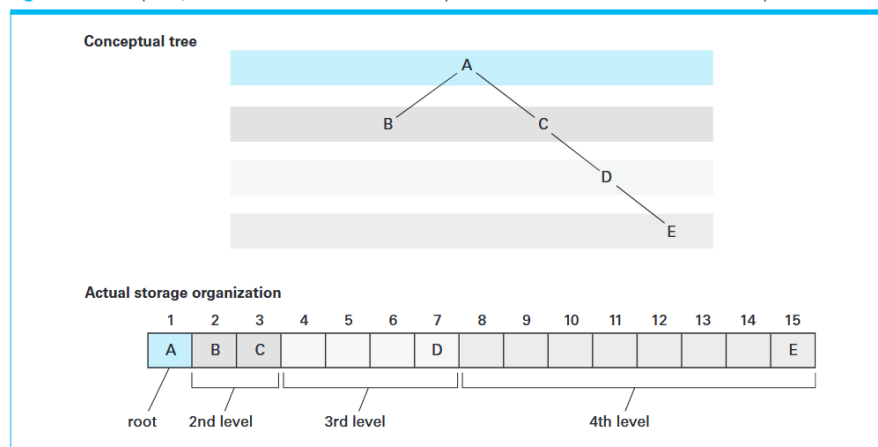


Figure 8.18 A sparse, unbalanced tree shown in its conceptual form and as it would be stored without pointers



Indexing Rule (连续存储索引规则)

Root node stored at cell 1

Left child of node at position $n \rightarrow$ cell $2n$

Right child of node at position $n \rightarrow$ cell $2n + 1$

Parent and Sibling (父节点与兄弟节点位置)

Parent of node at position $n \rightarrow \text{floor}(n / 2)$

Sibling of node at even position $n \rightarrow n + 1$

Sibling of node at odd position $n \rightarrow n - 1$

Manipulating Data Structures

A function `printList` can print a linked list abstractly.

-Each list entry has a value and a pointer (Next) to the next entry.

-The null pointer (None in Python) indicates the end of the list.

Figure 8.19 A function for printing a linked list

```
def PrintList(List):  
    CurrentPointer = List.Head  
    while (CurrentPointer != None):  
        print(CurrentPointer.Value)  
        CurrentPointer = CurrentPointer.Next
```

1. Show how the array below would be arranged in main memory when stored in row major order.

5	3	7
4	2	8
1	9	6

5 3 7 4 2 8 1 9 6

2. Give a formula for finding the entry in the i th row and j th column of a two-dimensional array if it is stored in column major order rather than row major order.

$$x + (r \times (j - 1)) + (i - 1)$$

3. In the Python, C, C++, Java, and C# programming languages, indices of arrays start at 0 rather than at 1. Thus the entry in the first row, fourth column of an array named `Array` is referenced by `Array[0][3]`. In this case, what address polynomial is used by the translator to convert references of the form `Array[i][j]` into memory addresses?

$$x + (c \times i) + j$$

4. What condition indicates that a linked list is empty?

When the head pointer points to a null pointer.

5. Modify the function in Figure 8.19 so that it stops printing once a particular name has been printed.

```
CurrentPointer = List.Head  
while (CurrentPointer != None):  
    print(CurrentPointer.Value)  
    if (CurrentPointer.Value == a particular name):  
        stop  
    CurrentPointer = CurrentPointer.Next
```

6. Based on the technique of this section for implementing a stack in a contiguous block of cells, what condition indicates that the stack is empty?

A stack is empty when the stack pointer points to the base of the stack.

7. Describe how a stack can be implemented in a high-level language in terms of a one-dimensional array.

A stack can be implemented using a one-dimensional array in which elements are stored in consecutive memory locations.

A stack pointer is used to keep track of the top of the stack.

Push and pop operations are performed by moving the stack pointer and accessing the array.

8. When a queue is implemented in a circular fashion as described in this section, what is the relationship between the head and tail pointers when the queue is empty? What about when the queue is full? How can one detect whether a queue is full or empty?

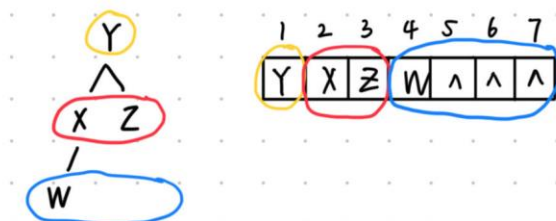
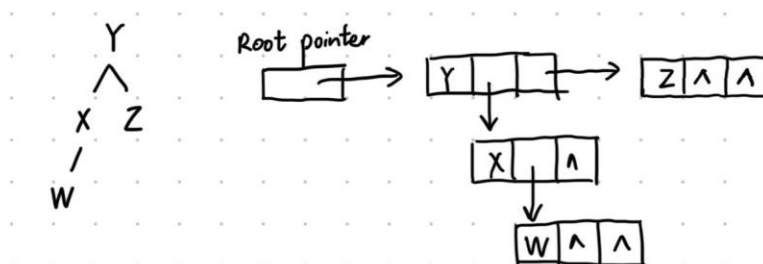
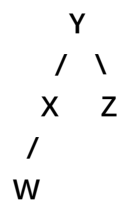
Empty: The queue is empty when the head pointer and tail pointer point to the same location.

Full: The queue is full when the tail pointer is about to advance to the position currently occupied by the head pointer.

Empty: $\text{head} == \text{tail}$

Full: $\text{next position of tail} == \text{head}$

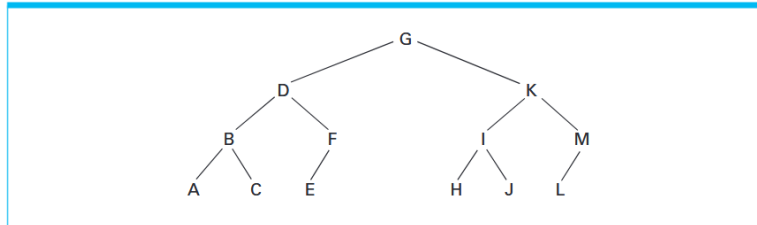
9. Draw a diagram representing how the tree below appears in memory when stored using the left and right child pointers, as described in this section. Then, draw another diagram showing how the tree would appear in contiguous storage using the alternative storage system described in this section.



8.4 A Short Case Study

search for the presence of an entry,
print the list in alphabetical order,
and insert a new entry

Figure 8.20 The letters A through M arranged in an ordered tree



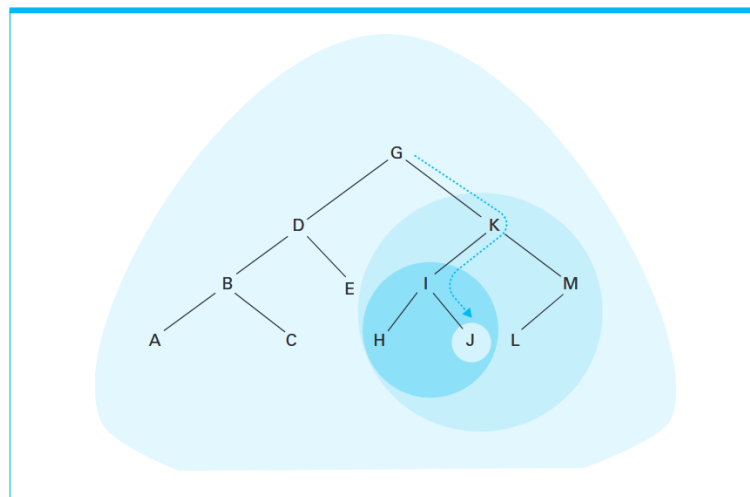
search

Tree represents the root node of the current subtree being searched.

Figure 8.21 The binary search as it would appear if the list were implemented as a linked binary tree

```
def Search(Tree, TargetValue):
    if (Tree is None):
        return None        # Search failed
    elif (TargetValue == Tree.Value):
        return Tree        # Search succeeded
    elif (TargetValue < Tree.Value):
        # Continue search in left subtree.
        return Search(Tree.Left, TargetValue)
    elif (TargetValue > Tree.Value):
        # Continue search in right subtree.
        return Search(Tree.Right, TargetValue)
```

Figure 8.22 The successively smaller trees considered by the function in Figure 8.21 when searching for the letter J



Search(G,J)

J>G

Search(K,J)

J<K

Search(I,J)

J>I

Search(J,J) return J

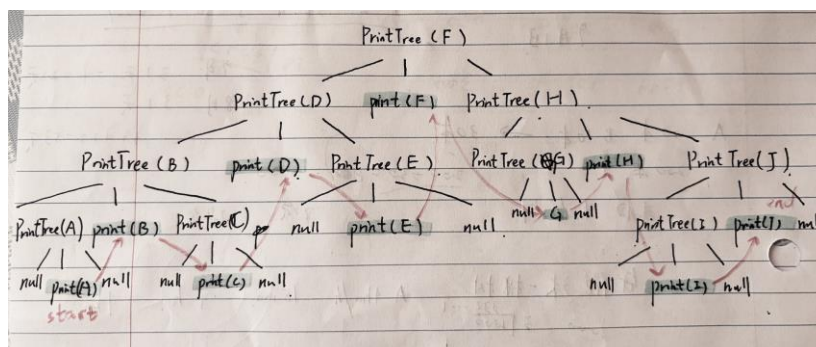
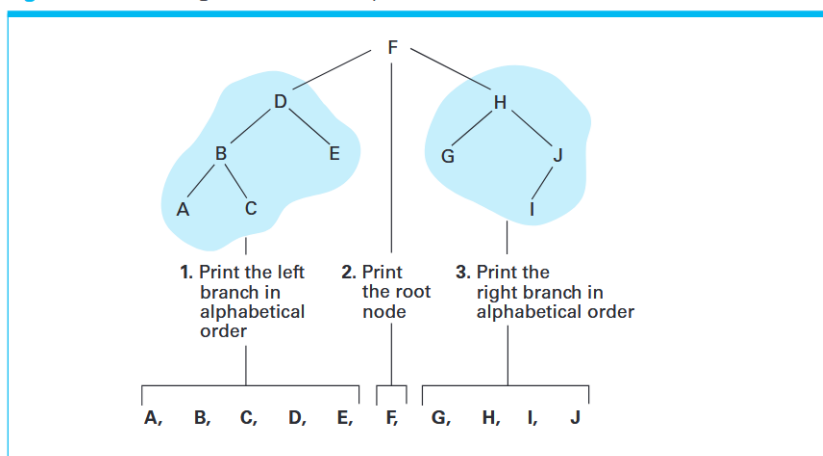
print

```
if (tree not empty):  
    print the left subtree in alphabetical order  
    print the root node  
    print the right subtree in alphabetical order
```

Figure 8.24 A function for printing the data in a binary tree

```
def PrintTree(Tree):  
    if (Tree != None):  
        PrintTree(Tree.Left)  
        print(Tree.Value)  
        PrintTree(Tree.Right)
```

Figure 8.23 Printing a search tree in alphabetical order

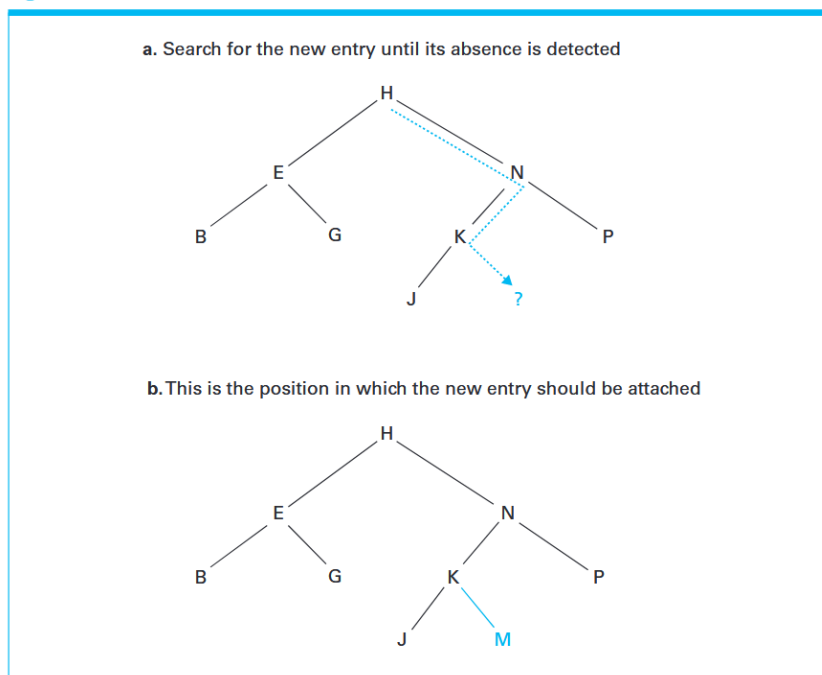


insert

Figure 8.26 A function for inserting a new entry in a list stored as a binary tree

```
def Insert(Tree, NewValue):  
    if (Tree is None):  
        # Create a new leaf with NewValue  
        Tree = TreeNode()  
        Tree.Value = NewValue  
    elif (NewValue < Tree.Value):  
        # Insert NewValue into the left subtree  
        Tree.Left = Insert(Tree.Left, NewValue)  
    elif (NewValue > Tree.Value):  
        # Insert NewValue into the right subtree  
        Tree.Right = Insert(Tree.Right, NewValue)  
    else:  
        # Make no change.  
    return Tree
```

Figure 8.25 Inserting the entry M into the list B, E, G, H, J, K, N, P stored as a tree



Insert(H,M)

M>H

Tree.Right = Insert(N,M)

M<N

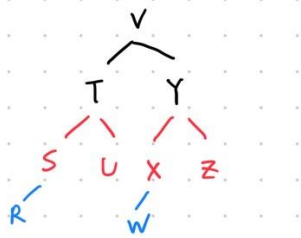
Tree.Left = Insert(K,M)

M>K

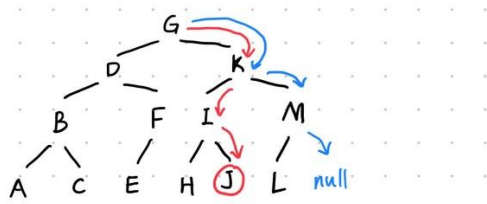
Tree.Right = Insert(null,M) -> Tree=TreeNode() Tree.Value=NewValue

1. Draw a binary tree that you could use to store the list R, S, T, U, V, W, X, Y, and Z for future searching.

R S T U V W X Y Z

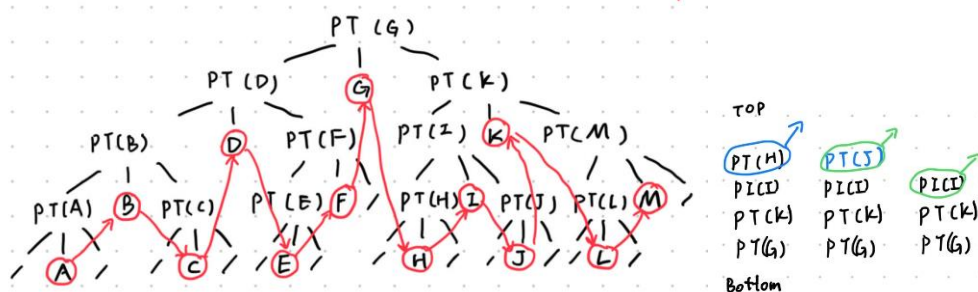


2. Indicate the path traversed by the binary search algorithm in Figure 8.21 when applied to the tree in Figure 8.20 in searching for the entry J. What about the entry P?

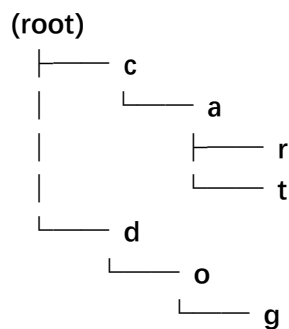


A B C D E F G H I J K L M N O P

3. Draw a diagram representing the status of activations of the recursive tree-printing algorithm in Figure 8.24 at the time node K is printed within the ordered tree in Figure 8.20.



4. Describe how a tree structure in which each node has as many as 26 children could be used to encode the correct spelling of words in the English language.



Each node can have up to 26 children, one for each letter.

A word is stored as a path from the root by following its letters.

The last node is marked as the end of the word.

8.5 Customized Data Types

A data type defines a set of values and the operations that can be performed on those values.

Primitive data types are built-in types provided by a programming language, such as integer, float, character, and Boolean.

User-Defined Data Types

user-defined data types User-defined data types are data types created by programmers using primitive data types as building blocks.

-Define a variable directly

```
struct {  
    char Name[25];  
    int Age;  
    float SkillRating;  
} Employee;
```

-Define a new data type

```
struct EmployeeType {  
    char Name[25];  
    int Age;  
    float SkillRating;  
};
```

-Declaring Variables of User-Defined Type (声明变量)

```
struct EmployeeType Employee1;
```

- Accessing Fields (访问字段)

```
Employee1.Age = 26;
```

-Multiple Instances (多个实例)

```
struct EmployeeType DistManager, SalesRep1, SalesRep2;
```

instance (实例)

An instance is an actual variable created from a data type.

Abstract Data Types

Traditional user-defined data types (such as structs in C) only define how data is stored. They do not define the operations that can be performed on the data.

abstract data type (ADT)

An abstract data type (ADT) is a user-defined data type that includes both data (representation) and operations (behavior).

Two Key Features of ADT

-Encapsulation (封装)

Programming languages provide syntax to define an ADT as a single unit combining data and functions.

-Information Hiding (信息隐藏)

The internal structure of the ADT is hidden from other parts of the program.

Why ADT is Important ADT improves reliability by preventing incorrect or unsafe access to data.

Stack Example

(1) Without ADT (没有 ADT)

```

struct StackType {
int StackEntries[20];
int StackPointer;
};
(2) With ADT (使用 ADT)
interface StackType {
    public int pop();
    public void push(int item);
    public boolean isEmpty();
    public boolean isFull();
}
StackType StackOne, StackTwo;
StackOne.push(25);

```

1. What is the difference between a data type and an instance of that type?

A data type defines a set of values and the operations that can be performed on those values.

An instance is an actual variable created from a data type.

2. What is the difference between a user-defined data type and an abstract data type?

A user-defined data type specifies how data is organized, while an abstract data type specifies both how data is organized and how it can be used.

3. Describe an abstract data type for implementing a list.

```

interface ListType {
    public void add(int item);
    public void delete(int item);
    public void modify(int oldItem, int newItem);
    public int find(int item);
}

```

4. Describe an abstract data type for implementing checking accounts.

```

interface AccountType {
    public void deposit(int amount);
    public void withdraw(int amount);
    public int getBalance();
    public boolean isOverdrawn();
}

```

8.6 Classes and Objects

Object-Oriented Paradigm (面向对象范式) The object-oriented paradigm organizes a program as a collection of interacting objects.

Object (对象) An object is an instance of a class that contains data and can perform operations.

Class (类) A class is a template used to create objects.

Classes are, in many respects, implementations or extensions of abstract data types (ADTs). An abstract data type defines what operations are allowed, while a class provides the implementation of those operations.

Figure 8.27 A stack of integers implemented in Java and C#

```
class StackOfIntegers implements StackType
{
    private int[] StackEntries = new int[20];
    private int StackPointer = 0;

    public void push(int NewEntry)
    { if (StackPointer < 20)
      StackEntries[StackPointer++] = NewEntry;
    }

    public int pop()
    { if (StackPointer > 0) return StackEntries[--StackPointer];
      else return 0;
    }

    public boolean isEmpty()
    { return (StackPointer == 0); }

    public boolean isFull()
    { return (StackPointer >= MAX); }
}
```

```
StackType StackOne = new StackOfIntegers();
```

```
StackOne.push(106);
```

```
OldValue = StackOne.pop();
```

Additional Features of Classes

(1) **Inheritance (继承)** Classes can inherit properties from other classes.

(2) **Constructors (构造函数)** Classes can include constructors to initialize objects.

(3) **Encapsulation (封装)** Classes support different levels of encapsulation to protect internal data.

1. In what ways are abstract data types and classes similar? In what ways are they different?

-Abstract data types and classes are similar in that both combine data and operations into a single unit and support encapsulation.

-They are different in that an abstract data type specifies the operations without implementation, while a class provides the implementation of those operations and may include additional features such as inheritance and constructors.

2. What is the difference between a class and an object?

A class is a template used to create objects.

An object is an instance of a class that contains data and can perform operations.

3. Describe a class that would be used as a template for constructing objects of type queue-of-integers.

```
class QueueOfIntegers {
    private int[] QueueEntries = new int[20];
    private int Front = 0;
    private int Rear = 0;

    public void enqueue(int item) {
        if (Rear < 20)
            QueueEntries[Rear++] = item;
    }

    public int dequeue() {
        if (Front < Rear)
            return QueueEntries[Front++];
        else
            return 0;
    }

    public boolean isEmpty() {
        return (Front == Rear);
    }

    public boolean isFull() {
        return (Rear >= 20);
    }
}
```

8.7 Pointers in Machine Language

immediate addressing 立即数寻址

Op-code 2 loads a register with data given directly in the instruction.

direct addressing 直接寻址

Op-code 1 loads a register with data from a specific memory address.

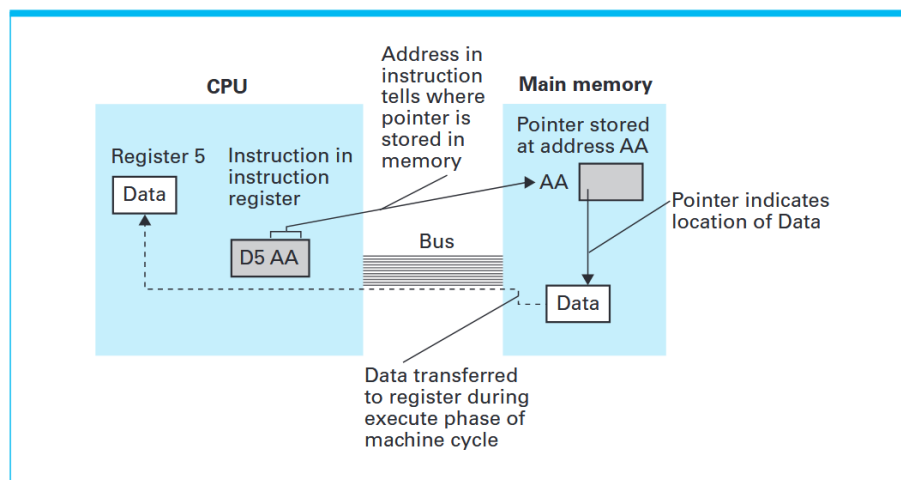
indirect addressing 间接寻址

We need an instruction that loads data using a pointer (address of an address)

New op-code D (memory-based pointer)

DRXY means: load register R with the contents of the memory cell whose address is stored at XY.

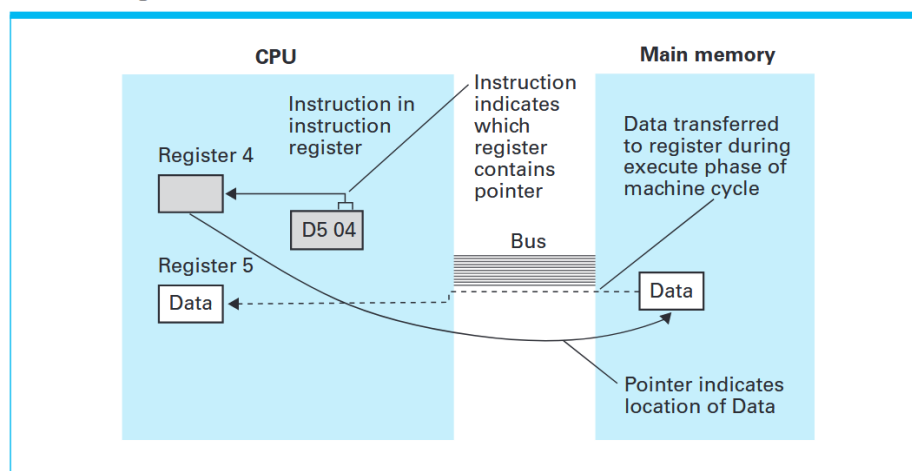
Figure 8.28 Our first attempt at expanding the machine language in Appendix C to take advantage of pointers



D5AA

1. 读取 AA → 得到栈顶地址
2. 再访问该地址 → 取出数据
3. 放入寄存器 5

Figure 8.29 Loading a register from a memory cell that is located by means of a pointer stored in a register



New design: pointer in register

DR0S means: load register R with data pointed to by register S.

1. 从寄存器 S 取地址
2. 访问该地址
3. 取数据 → 放入 R

Push operation (op-code E)

EROS stores data from register R into the memory pointed to by S.

把寄存器 R 的数据写到 S 指向的位置

1. Suppose the machine language described in Appendix C has been extended as suggested at the end of this section. Moreover, suppose register 8 contains the pattern DB, the memory cell at address DB contains the pattern CA, and the cell at address CA contains the pattern A5. What bit pattern will be in register 5 immediately after executing each of the following instructions?

R8	addressDB	addressCA	R5
DB	CA	A5	

a. 25A5

A5 直接给数据

b. 15CA

A5 找一次内存

c. D508

CA 用寄存器当指针（只找一次）

2. Using the extensions described at the end of this section, write a complete machine language routine to perform a pop operation. Assume that the stack is implemented as shown in Figure 8.12, the stack pointer is in register F, and the top of the stack is to be popped into register 5.

找到栈顶数据放到 R5

D50F

移动 stack pointer

2AFF ; Load -1 into register A

5FAF ; Add A to F ($F = F - 1$)

3. Using the extensions described at the end of this section, write a program to copy the contents of five contiguous memory cells starting at address A0 to the five cells starting at address B0. Assume your program starts at address 00.

00: 2AA0 ; A = A0

01: 2BB0 ; B = B0

02: 2C05 ; C = 5

03: D50A ; R5 = Memory[A]

04: E50B ; Memory[B] = R5

05: 2201 ; R2 = 1

06: 5AA2 ; A = A + 1

07: 5BB2 ; B = B + 1

08: 2DFF ; D = -1
09: 5CCD ; C = C - 1

0A: BC0D ; if C == 0 jump to 0D
0B: B003 ; unconditional jump to 03 (用 R0 == R0)

0D: C000 ; HALT

4. In the chapter, we introduced a machine instruction of the form DR0S. Suppose we extended this form to DRXS, meaning “Load register R with the data pointed to by the value in register S plus the value X.” Thus the pointer to the data is obtained by retrieving the value in register S and then incrementing that value by X. The value in register S is not altered. (If register F contained 04, then the instruction DE2F would load register E with the contents of the memory cell at address 06. The value of register F would remain 04.) What advantages would this instruction have?

DRXS allows accessing elements with an offset without modifying the pointer.

A = base address

DR0A → A[0]

DR1A → A[1]

DR2A → A[2]

No need to increment and restore the pointer.

Pointer remains unchanged

X 是个常数

What about an instruction of the form DRTS—meaning “Load register R with the data pointed to by the value in register S incremented by the value in register T”?

T 是个寄存器 存的东西可以变

Fully dynamic indexing

The offset(偏移量) can be stored in a register.

DRTS allows accessing A[i]

S = base

T = i

可以用来写循环

i = 0 → n-1

R = Memory[base + i]

i++